

ARTIFICIAL INTELLIGENCE

Lecture Set 03

Problem Solving by Searching

Dr. Maria Siddiqua
Assistant Professor
Department of AI & DS
FAST-NUCES Karachi
maria.siddiqua@nu.edu.pk

Course Contents

1. Introduction to AI
2. Agents and Environments
- **3. Problem Solving by Searching**
4. Uninformed Search
5. Informed Search
6. Local Search
7. Constraint Satisfaction Problems
8. Adversarial Search
9. Logical Agents
10. First-order Logic
11. Uncertain Knowledge and Reasoning
12. Supervised, Unsupervised, and Reinforcement Learning
13. Recent Trends in AI and Applications of AI Algorithms

Table of Contents

- Problem Solving Agents
- Problem Formulation
- Graph Theory
- Undirected graph
- Directed Graph
- Weighted Graph
- Adjacency Matrix
- Path, Path Cost and Shortest Path
- Do it by yourself
- Solving Problems by Searching
- Basic Terminology
- How to Solve a Problem?
- Categories of Problems in AI

Table of Contents

- Single-Agent Path-Finding Problems
- Two-Player Games
- Constraint-Satisfaction Problems
- Problem Space
- Problem Instance
- Representing States
- TIC-TAC-TOE
- 8 – Puzzle
- 8 Queens Problem
- Real World Problems
- Search Strategies
- Forward Search
- Backward Search
- Evaluating Search Strategies

Previously we studied: Basic Agent Types

Agents can be grouped into six classes based on their degree of perceived intelligence and capacity.

1. **Table Driven Agents** (look-up table of percept-action pairs)
2. **Simple Reflex Agents** (present condition-based decision)
3. **Model Based Reflex Agents** (memory-based decision)
4. **Goal Based Agents**
5. **Utility Based Agents** (Maximize the profit)
6. **Learning Based Agents** (learn from experience)

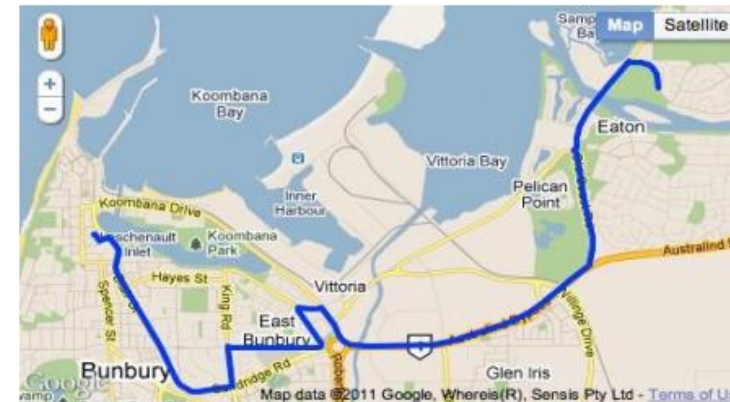
Reflex agents v. Problem solving agents

A simple reflex agent is one that selects an action based on the current percept, and ignores the rest of the percept history.



A problem-solving agent must plan ahead. It will consider a sequence of actions that form a path to a goal state. The computational process that it undertakes is called search.

**Achieve intelligence by
(searching) a solution!**



Problem Solving Agents

- Agent that tries to **come up** with a **sequence of actions** by **performing** some kind of **search algorithm** in the background that will **bring** the environment **into** a desired state/**goal**.
- The use of search requires an abstract formulation of the problem and the available steps to construct solutions.
- We need to first formulate the problem.

Problem Formulation

1.State



2.Initial State



3.Actions



4.Goal Test



5.Path Cost



6.Solution

A State Space

Set of all possible states where you can be.

A Start State: The state from where the search begins.

A Goal Test: A function that looks at the current state returns whether or not it is the goal state.

Solution to a search problem is a sequence of actions, called the **plan** that transforms the start state to the goal state.

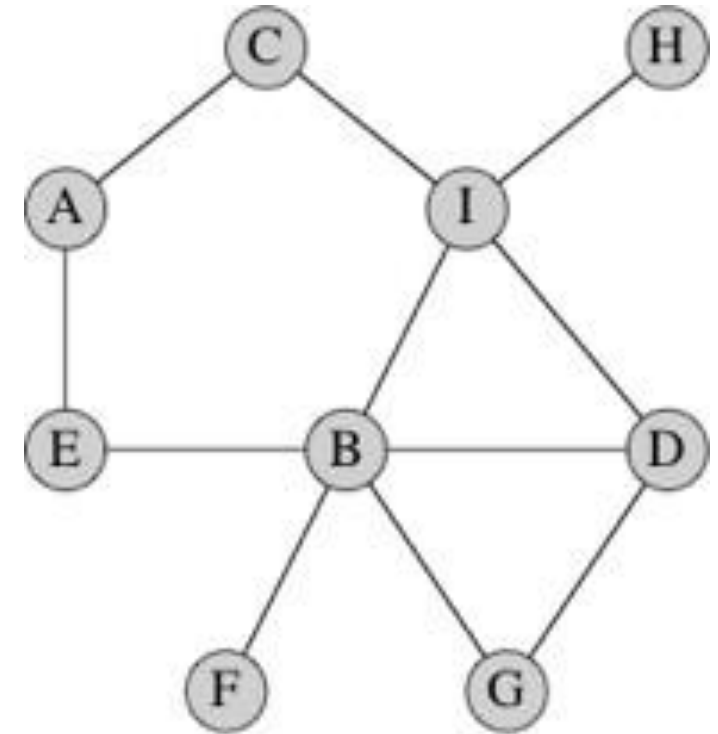
This plan is achieved through search algorithms

Graph Theory

- A Graph is the pair of sets V and E . Where V is the set of vertices (nodes) and E is the set of edges.

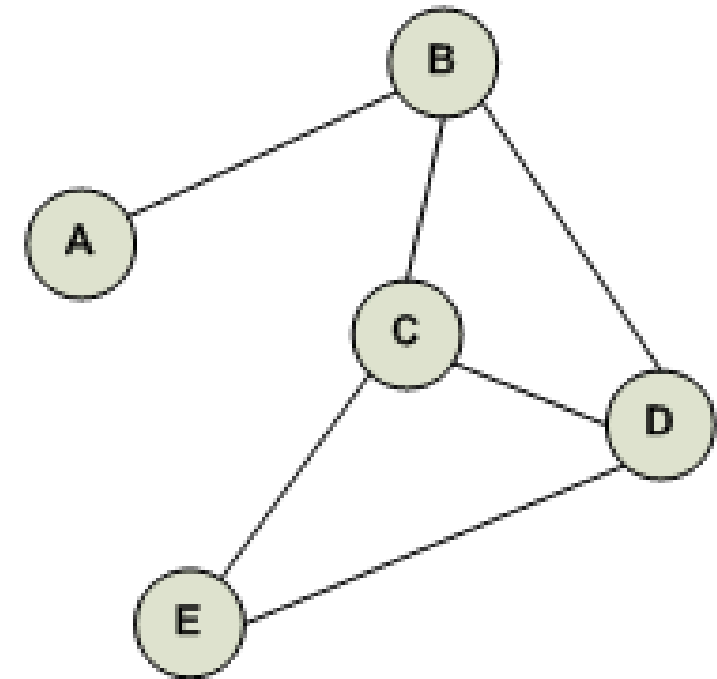
Mathematically,

- $G = (V, E)$
- $V = \{A, B, C, D, E, F, G, H, I\}$
- $E = \{AC, AE, CI, EB, BD, BF, BG, BI, DI, DG, HI\}$



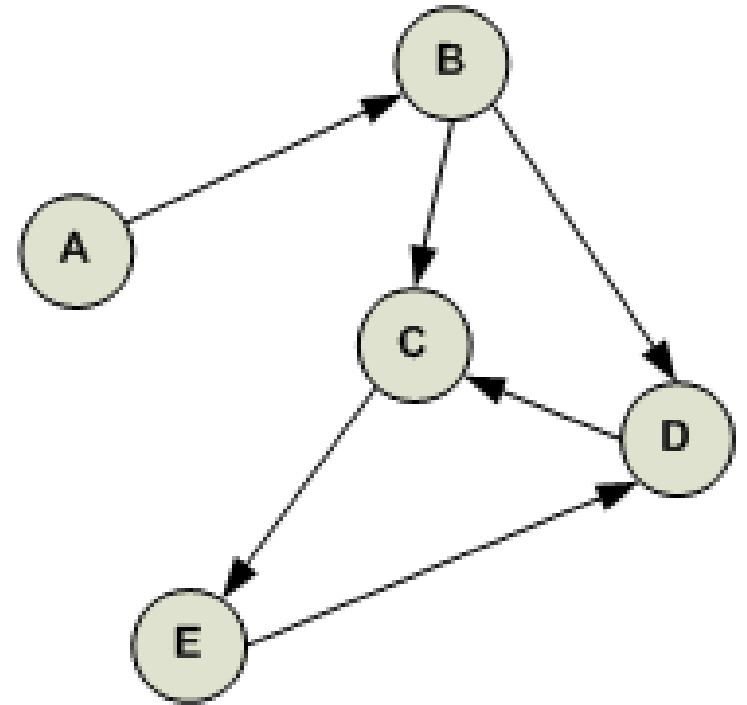
Undirected Graph

- Undirected graph contains edges but the edges are not directed ones.
- Example:
- $G = (V, E)$
- $V = \{A, B, C, D, E\}$
- $E = \{AB, BC, BD, CD, CE, DE\}$



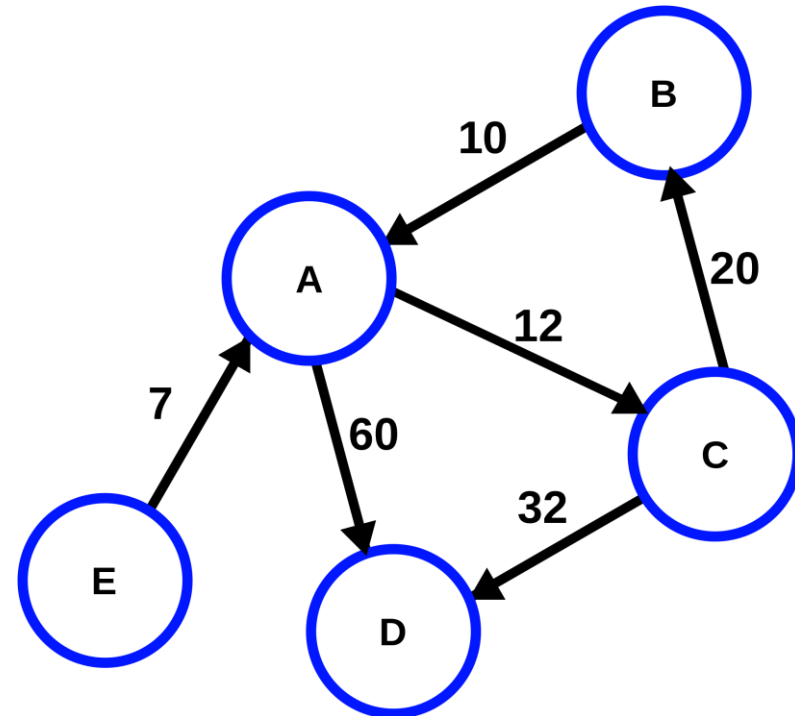
Directed Graph

- In a directed graph, each edge has a direction.
- Example:
- $G = (V, E)$
- $V = \{A, B, C, D, E\}$
- $E = \{AB, BC, BD, CE, DC, ED\}$



Weighted Graph

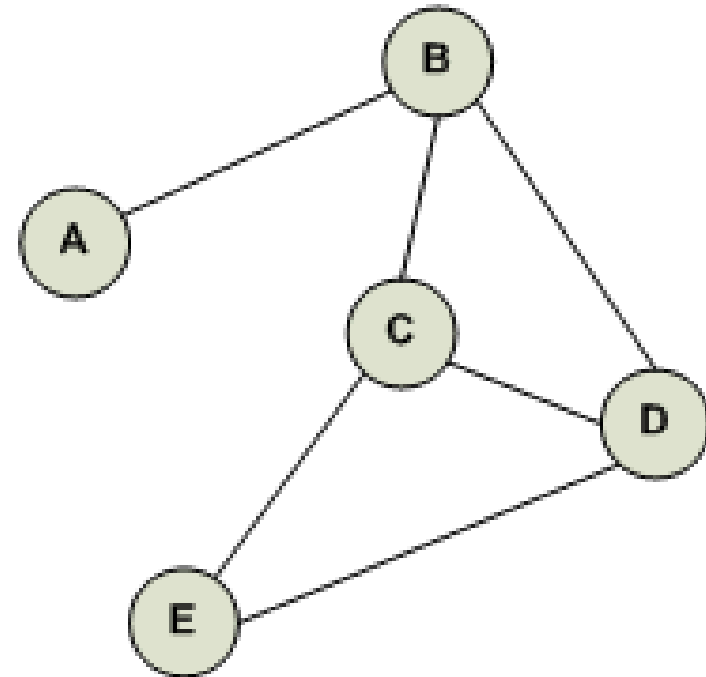
- A graph in which each of the edge is associated with a real number (weight) is called as weighted graph. Both undirected and directed graphs can be weighted.
- Example:
- $G = (V, E)$
- $V = \{A, B, C, D, E\}$
- $E = \{AC, AD, BA, CB, CD, EA\}$
- $\text{Weights} = \{12, 60, 10, 20, 32, 7\}$



Adjacency Matrix

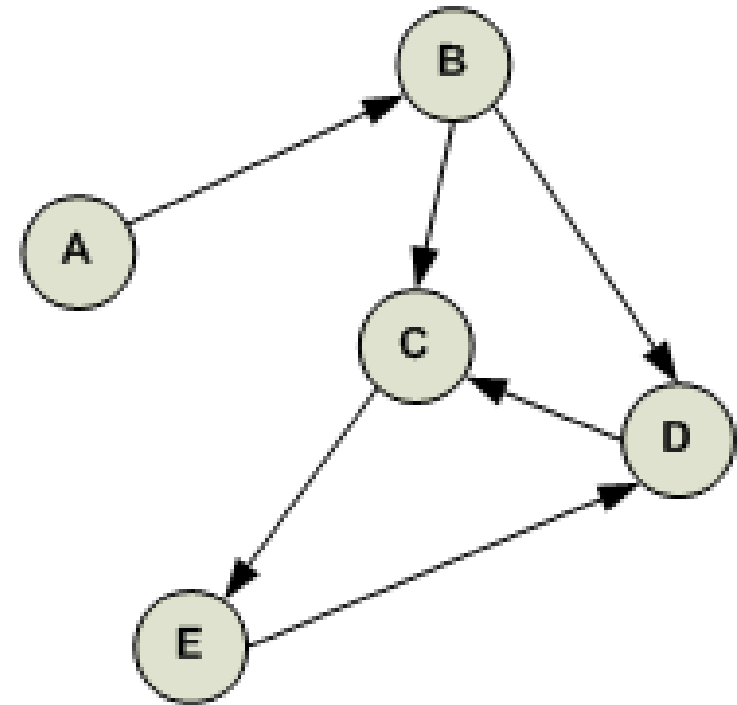
- The adjacency matrix, sometimes also called the connection matrix, of a simple labeled graph is a matrix with rows and columns labeled by graph vertices, with a 1 or 0 in position (v_i, v_j) according to whether v_i and v_j are adjacent or not.

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
<i>A</i>	0	1	0	0	0
<i>B</i>	1	0	1	1	0
<i>C</i>	0	1	0	1	1
<i>D</i>	0	1	1	0	1
<i>E</i>	0	0	1	1	0



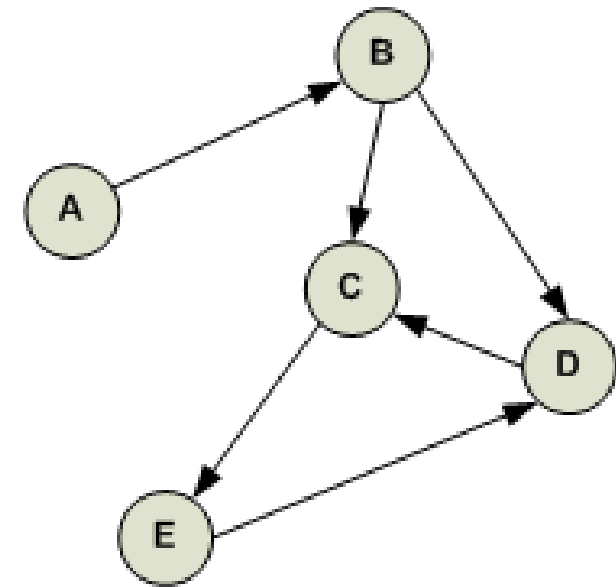
Adjacency Matrix

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
<i>A</i>	0	1	0	0	0
<i>B</i>	0	0	1	1	0
<i>C</i>	0	0	0	0	1
<i>D</i>	0	0	1	0	0
<i>E</i>	0	0	0	1	0



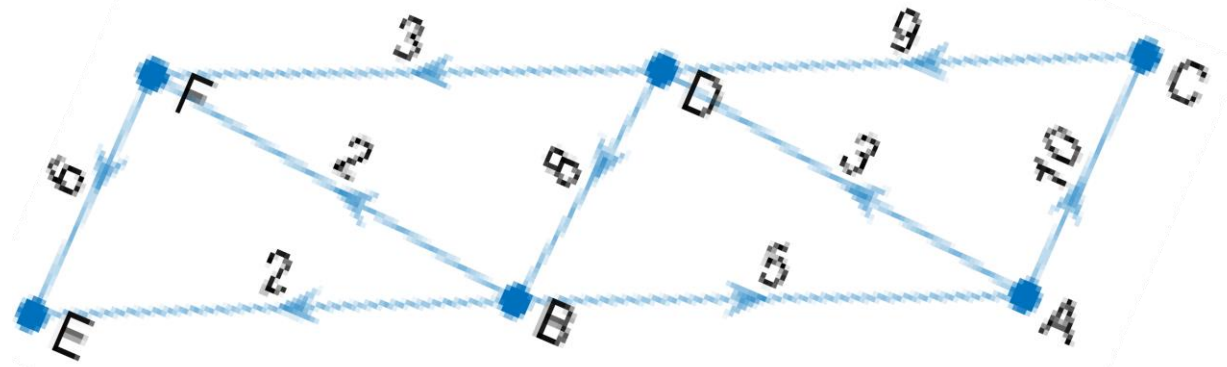
Path, Path Cost and Shortest Path

- Path: The sequence of all the edges connecting from source node to destination node.
 - Path Cost: It is the sum of all the consequent edge weights in a path.
 - Shortest Path: The path with minimum path cost.
-
- Path from A to E: $A \rightarrow B \rightarrow C \rightarrow E$
 - Path from C to D: $C \rightarrow E \rightarrow D$



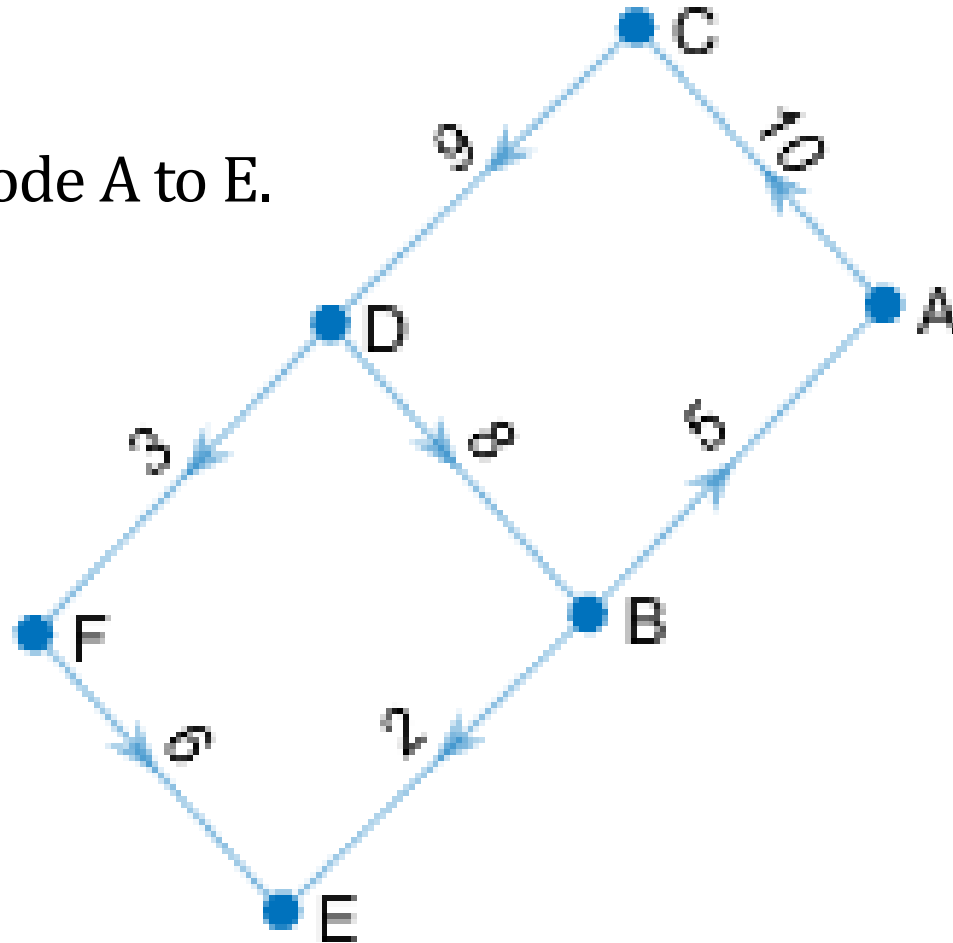
Path, Path Cost and Shortest Path

- Paths from A to F:
- Path 1 = $A \rightarrow C \rightarrow D \rightarrow F$ (Sum of weights = $10 + 9 + 3 = 22$)
- Path 2 = $A \rightarrow C \rightarrow D \rightarrow B \rightarrow F$ (Sum of weights = $10 + 9 + 8 + 2 = 29$)
- Path 3 = $A \rightarrow D \rightarrow F$ (Sum of weights = $3 + 3 = 6$)
- Path 4 = $A \rightarrow D \rightarrow B \rightarrow F$ (Sum of weights = $3 + 8 + 2 = 13$)
- Path 3 is the shortest path from node A to F.



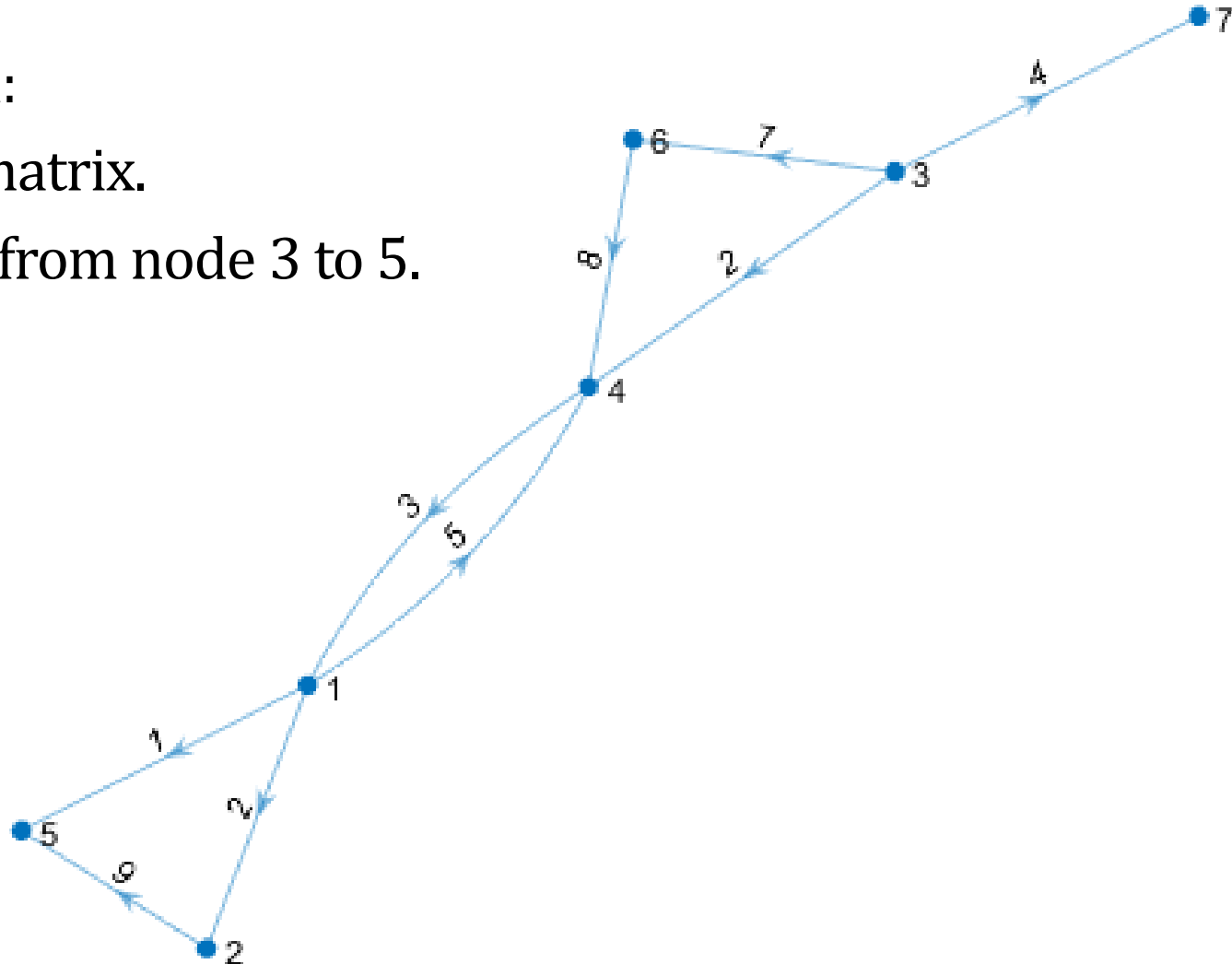
Do it by yourself

- For the following graph:
 - Create Adjacency matrix.
 - Find shortest path from node A to E.



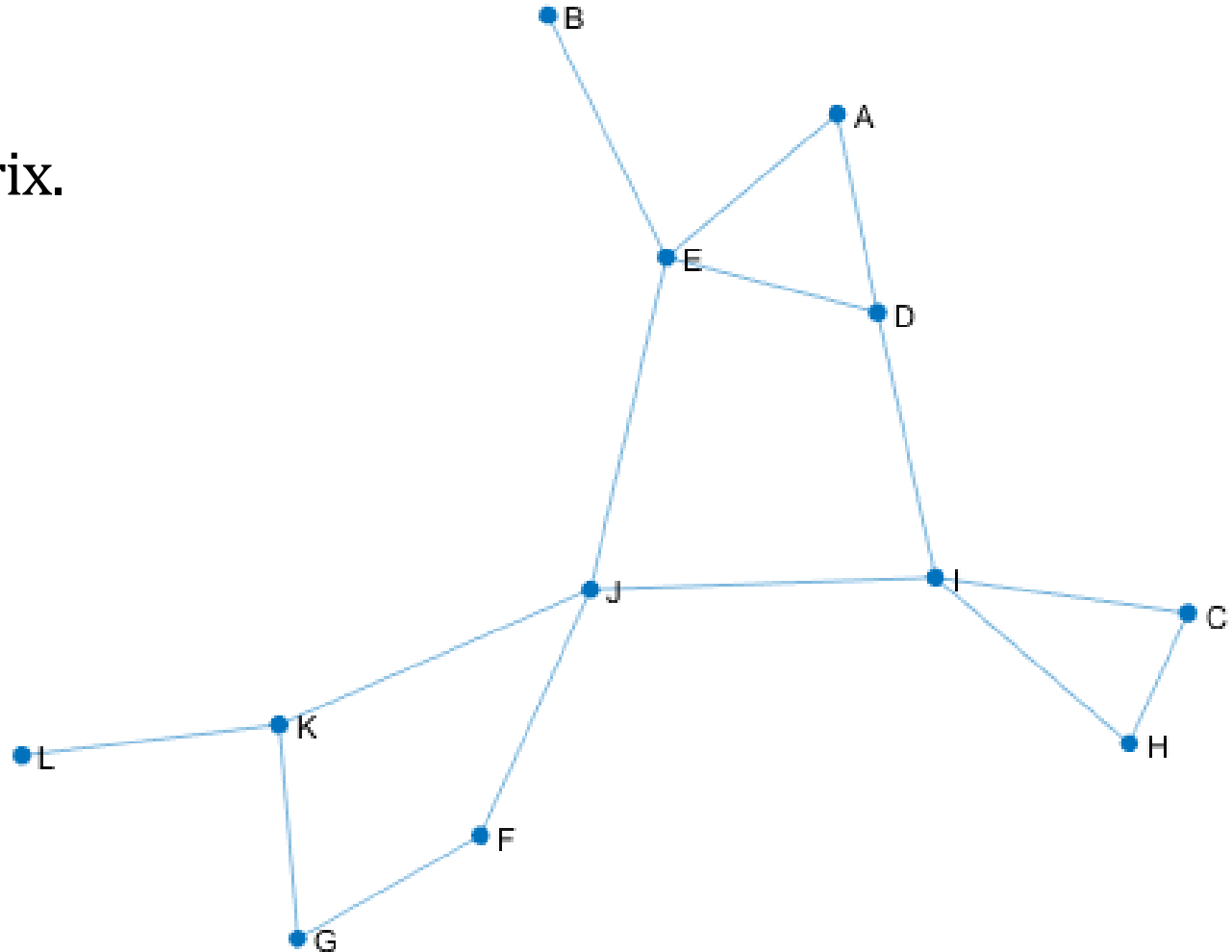
Do it by yourself

- For the following graph:
 - Create Adjacency matrix.
 - Find shortest path from node 3 to 5.



Do it by yourself

- For the following graph:
 - Create Adjacency matrix.



Solving Problems by Searching

- Reflex Agents are limited in what they can do because:
 - *Their actions are determined only by the current percept.*
 - *Have no knowledge of what their actions do.*
 - *Do not know what they are trying to achieve.*
- Goal-based agent are called as **problem-solving agent**.
- Problem-solving agents decide what to do by finding sequences of actions that lead to desirable states.

Solving Problems by Searching

- The problem-solving agent decides what sequence of actions will lead it to the desired goal.
- The **process of finding** such a sequence of actions is called as **Search**.
- A search algorithm takes a problem as input and returns a **solution** in the form of an action sequence.
- Once a solution is found, the actions it recommends can be carried out.
- This is called the **execution** phase.

Basic Terminology

- **Problem:** A collection of information that the agent will use to decide what to do.
- **State:** A state is a situation that an agent can find itself in.
- **Initial State:** The starting state that the agent knows itself to be in.
- **Goal State:** The final state, the agent is trying to reach.
- **Operators:** The set of possible actions available to the agent. Operators transform agent from one state to another.

Basic Terminology

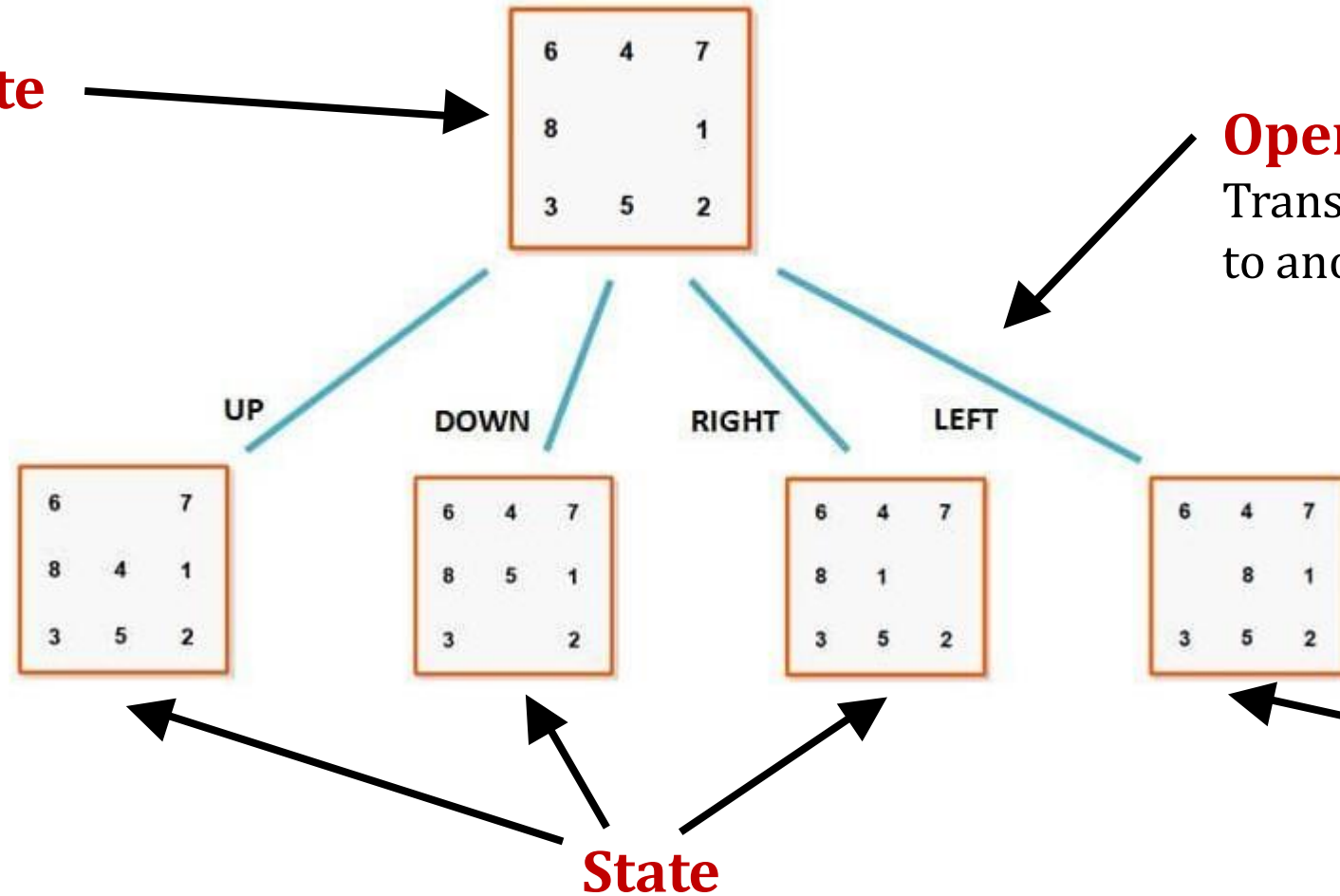
- **State Space:** The set of all states reachable from the initial state by any sequence of actions.
- It is a graph whose nodes are the set of all states, and whose links are the actions that transform one state into another.
- **Path:** A path in the state space is simply any sequence of actions leading from one state to another.

Basic Terminology

State Space

All states + All operations in graph

Initial State



Operation/Action

Transforms one state in to another

Goal State

State

Basic Terminology

5	4	
6	1	8
7	3	2

Initial State

1	2	3
8		4
7	6	5

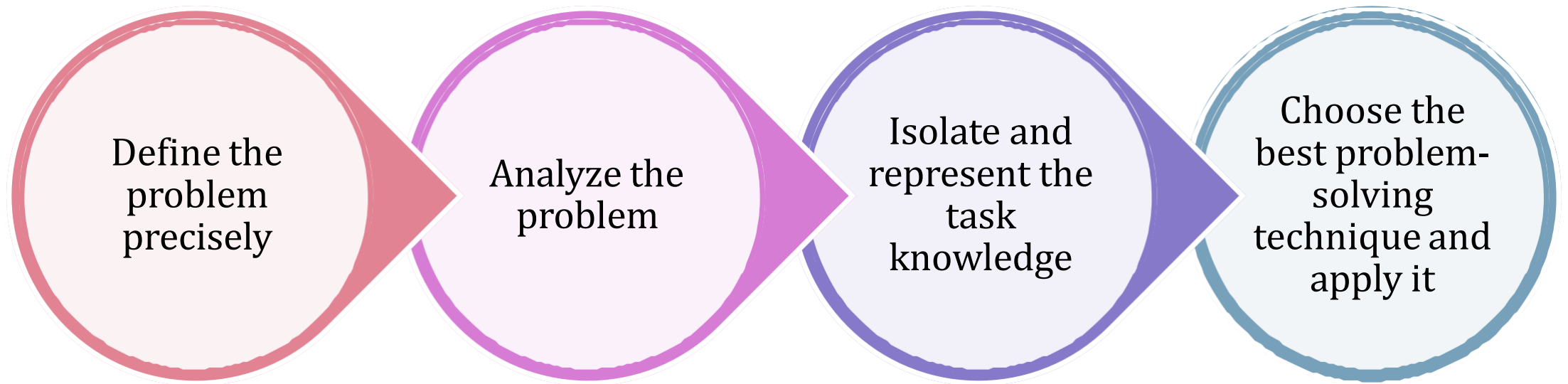
Goal State

Basic Terminology

- **Search Tree:** A search tree is a tree in which the root node is the initial state and the set of children for each node consists of the states reachable by taking any action.
- **Search Node:** A search node is a node in the search tree.
- **Action:** An action is something that the agent can choose to do.
- **Successor Function:** A successor function describes the agent's options: given a state, it returns a set of (action, state) pairs, where each state is the state reachable by taking the action.
- **The Branching Factor:** The branching factor in a search tree is the number of actions available to the agent.

How to Solve a Problem?

- In order to solve a problem:



Categories of Problems in AI

- There are 3 general categories of problems in AI:
 1. Single-agent path-finding problems
 2. Two-player games
 3. Constraint-satisfaction problems

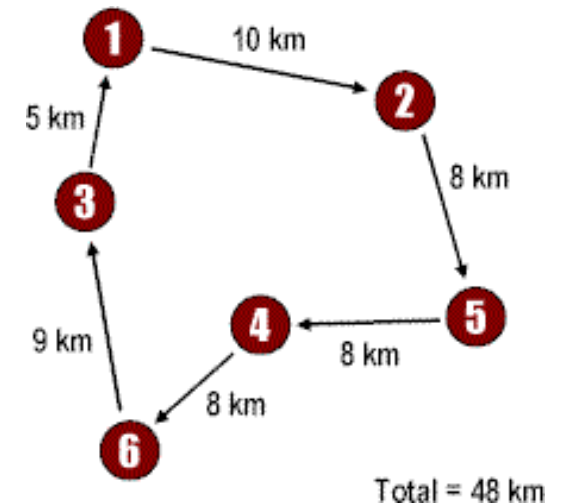
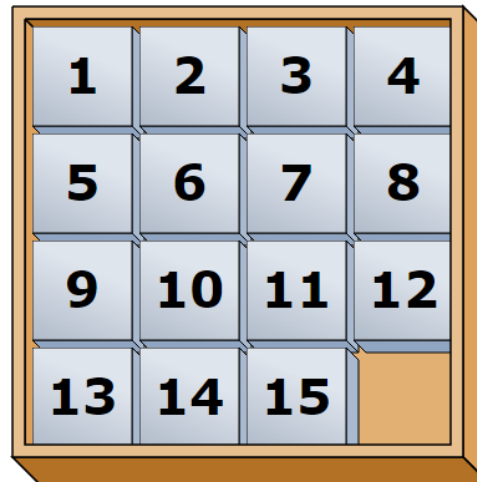
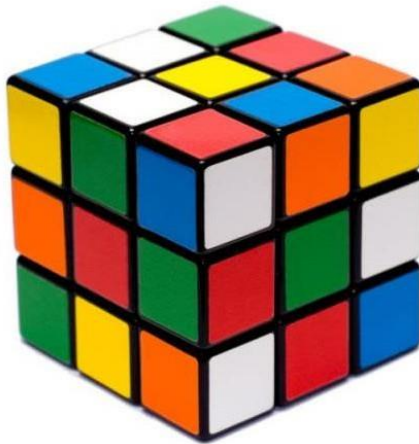
Single-Agent Path-Finding Problems

- In these problems, in each case, we have a single problem-solver making the decisions.
- The task is to find a sequence of primitive steps that take us from the initial location to the goal location.

Single-Agent Path-Finding Problems

Examples:

- Rubik's Cube.
- Sliding-Tile puzzle.
- Navigation - Travelling Salesman Problem.

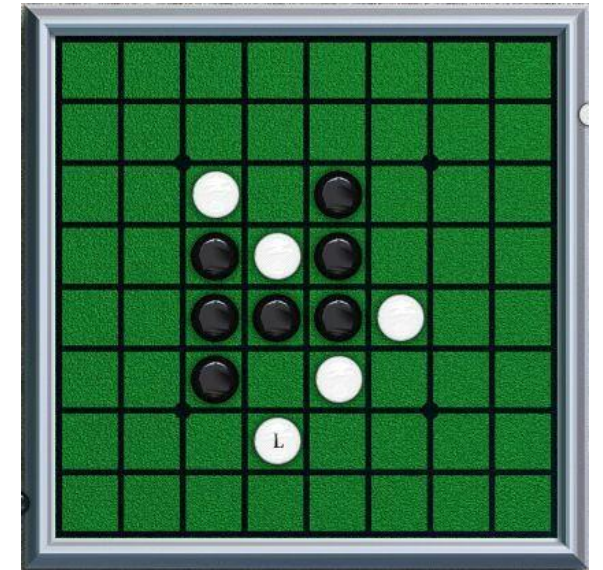


Two-Player Games

- In a two-player game, one must consider the moves of an opponent.
- The ultimate goal is a strategy that will guarantee a win whenever possible.
- Two-player perfect information have received the most attention of the researchers till now. But, nowadays, researchers are starting to consider more complex games, many of them involve an element of chance.

Two-Player Games

- The best Chess, Checkers, and Othello players in the world are computer programs!



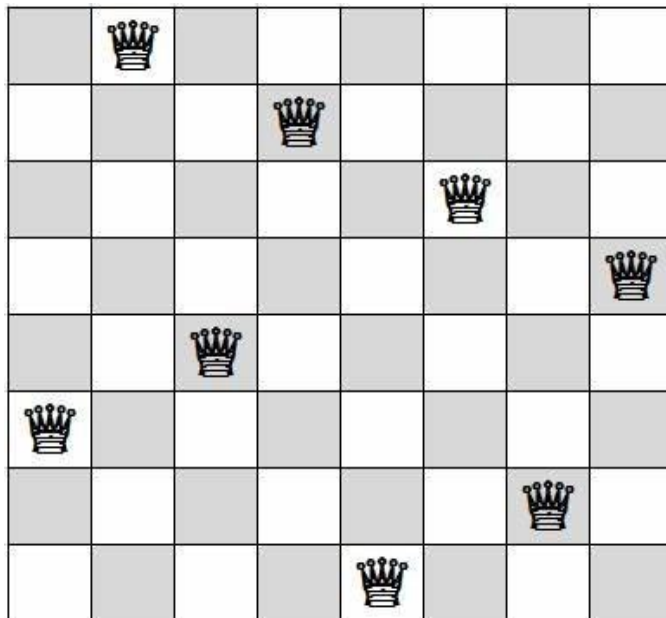
Constraint-Satisfaction Problems

- In these problems, we also have a single-agent making all the decisions, but here we are not concerned with the sequence of steps required to reach the solution, but simply the solution itself.
- The task is to identify a state of the problem, such that all the constraints of the problem are satisfied.

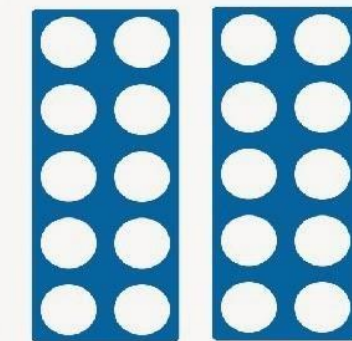
Constraint-Satisfaction Problems

■ Famous Examples:

- *Eight Queens Problem* (no two queens share the same row, column, or diagonal)
- *Number Partitioning*.



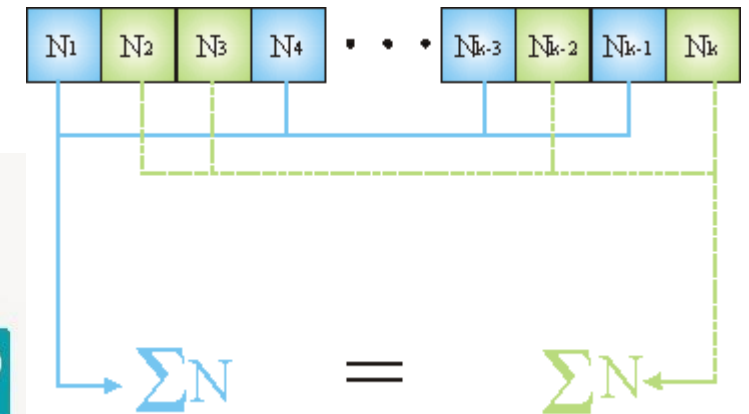
26 =



Two Tens &



Six Units



Problem Space

- A problem space consists of a **set of states** of a problem and a **set of operators** that change the state.
 - **State:** *a symbolic structure that represents a single configuration of the problem in a sufficient detail to allow problem solving to proceed.*
 - **Operator :** *a function that takes a state and maps it to another state.*

Problem Instance

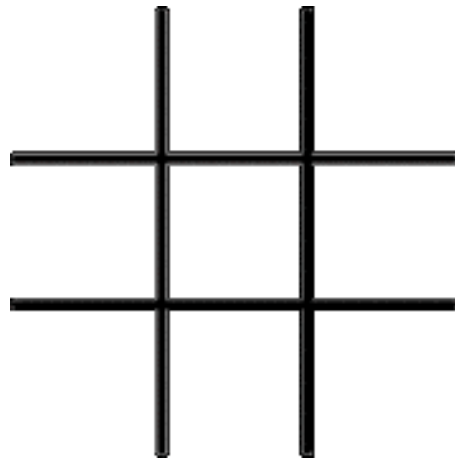
- A problem (or, more precisely, a problem instance) consists of:
 - *A problem space*
 - *An initial state*
 - *A set of goal states*
 - *A solution:*
 - A sequence of operator applications leading from the initial state to a goal state.
 - A path through the state space from initial to final state.

Representing States

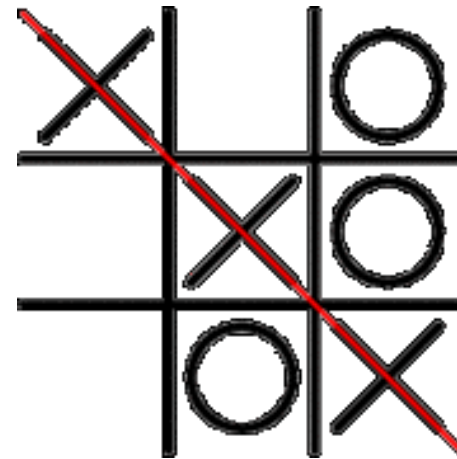
- The state-space representation of a problem is a triplet **(I, O, G)**
 - *Where: I - initial state, O - a set of operators on states, G - goal states*
- A state space can be organized as a graph:
 - *Nodes: States in the space*
 - *Edges: Actions/Operations*
- A solution is a path from the initial state to a goal state.
- The size of a problem is usually described in terms of the number of states (or the size of the state space) that are possible.
- Chess has about 10^{120} states in a typical game.

TIC-TAC-TOE

- Given an initial configuration of 9 cells on a 3 x 3 board, place crosses and zeros such that to achieve a desired goal configuration of the board.



Initial State

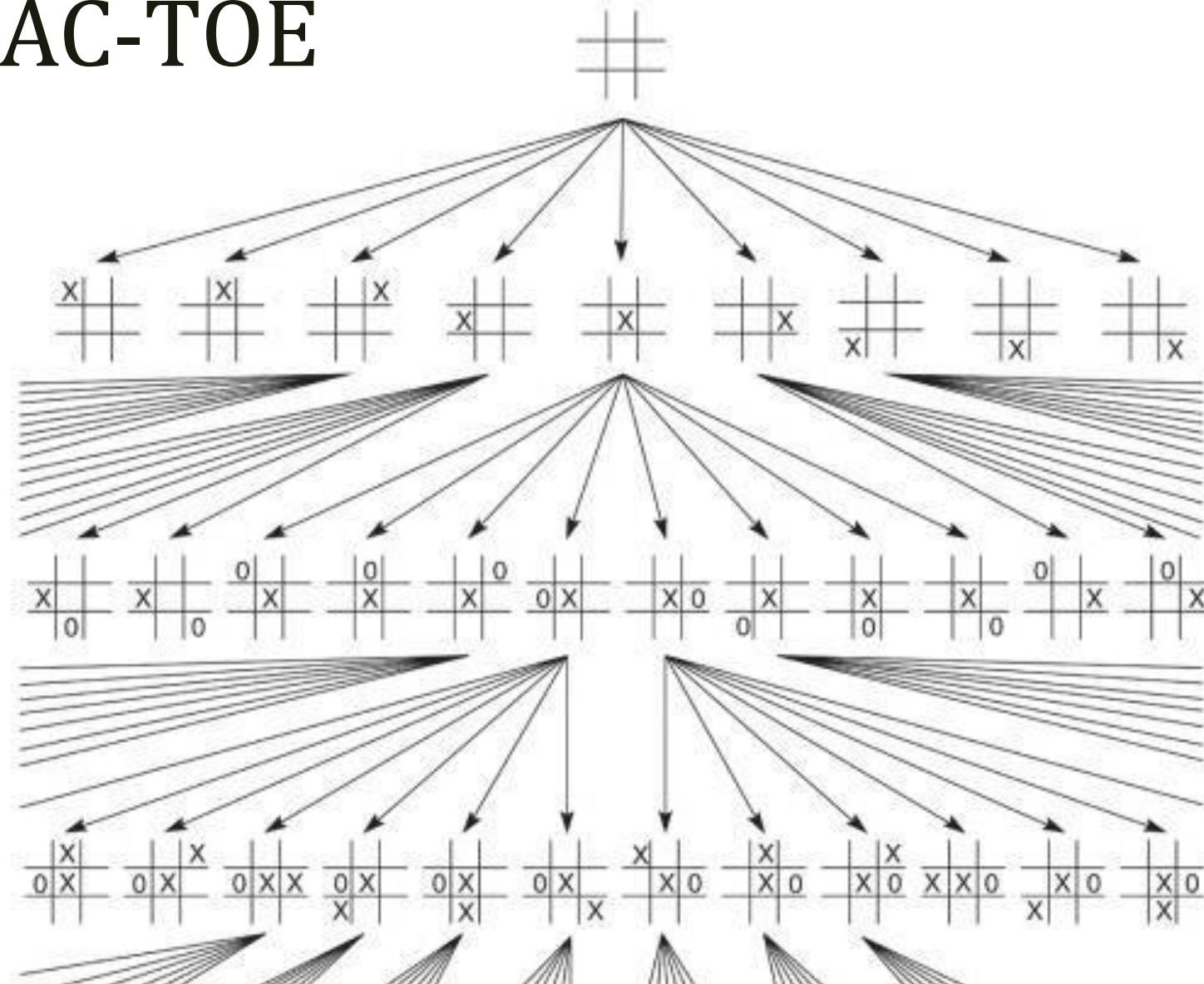


Goal State

TIC-TAC-TOE

- **State:** 3 x 3 array configuration of board with crosses and zeros.
- **Operators:** Placing a cross, placing a zero.
- **Initial State:** A particular configuration of the board.
- **Goal:** Three zeros or crosses; horizontally, vertically or diagonally.
- **Path Cost:** Each step costs 1, so the path cost is just the length of the path.

TIC-TAC-TOE



8 - Puzzle

- Given an initial configuration of 8 numbered tiles on a 3 x 3 board, move the tiles in such a way so as to produce a desired goal configuration of the tiles.

5	4	
6	1	8
7	3	2

Start State

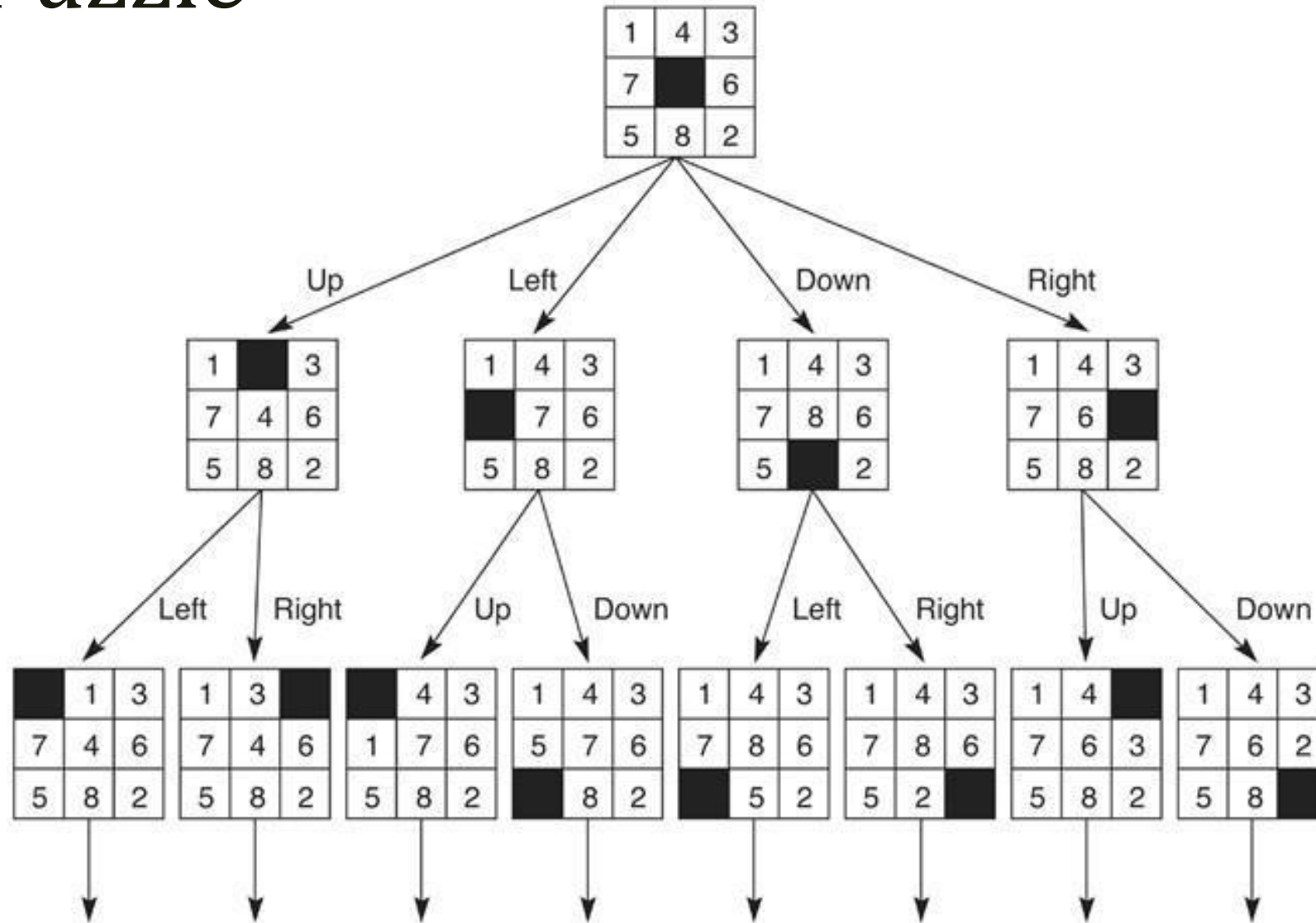
1	2	3
8		4
7	6	5

Goal State

8 - Puzzle

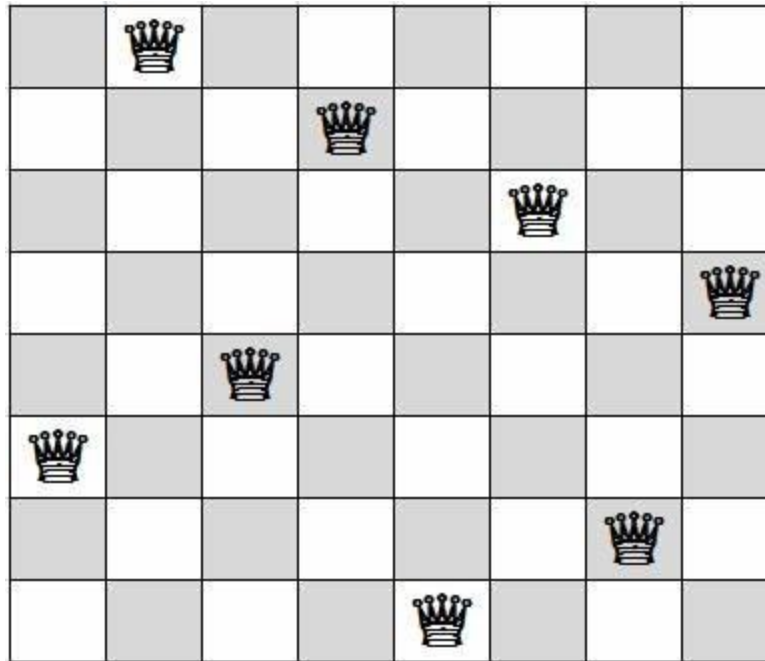
- **State:** 3 x 3 array configuration of the tiles on the board.
- **Operators:** Blank moves Left, Right, Up or Down.
- **Initial State:** A particular configuration of the board.
- **Goal:** A particular configuration of the board.
- **Path Cost:** Each step costs 1, so the path cost is just the length of the path.

8 - Puzzle



8 Queens Problem

- The goal of 8-Queen problem is to place eight queens on a chessboard such that no queen attacks any other. (A queen attacks any piece in the same row, column or diagonal).



8 Queens Problem

- **State:** Any arrangement of 0 to 8 queens on board.
- **Operators:** Add a queen to board.
- **Initial State:** No queen on the board.
- **Goal:** 8 queens on board, none attacked.
- **Path Cost:** Each step costs 1, so the path cost is just the length of the path.

Real World Problems

Route Finding Problem



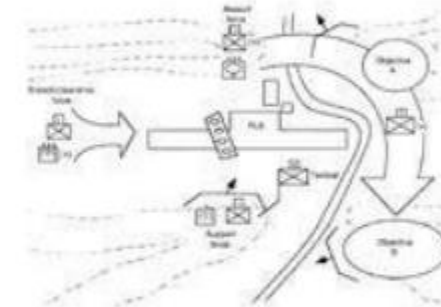
Car
Navigation



Airline travel
planning



Routing in Computer
networks



Military operation
planning

- **States**
 - locations
- **Initial state**
 - starting point
- **Successor function (operators)**
 - move from one location to another
- **Goal test**
 - arrive at a certain location
- **Path cost**
 - may be quite complex
 - money, time, travel comfort, scenery,

Real World Problems

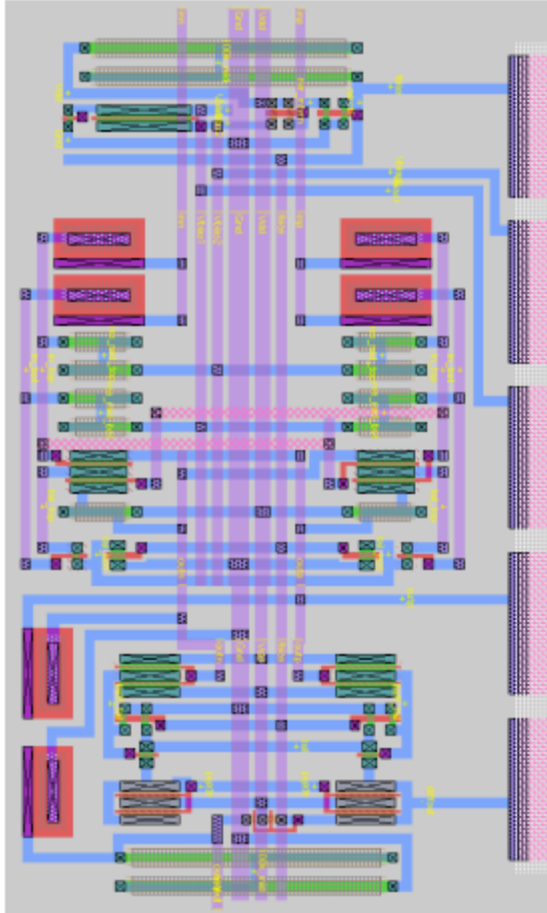
Travel Salesperson Problem



- **States**
 - locations / cities
 - illegal states
 - each city may be visited only once
 - visited cities must be kept as state information
- **Initial state**
 - starting point
 - no cities visited
- **Successor function (operators)**
 - move from one location to another one
- **Goal test**
 - all locations visited
 - agent at the initial location
- **Path cost**
 - distance between locations

Real World Problems

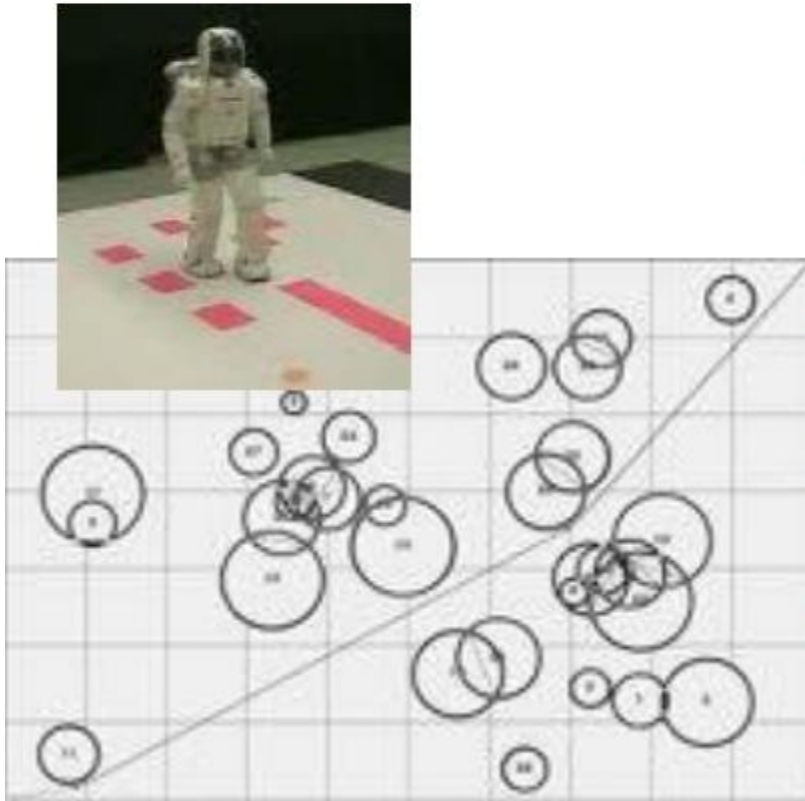
VLSI layout Problem



- **States**
 - positions of components, wires on a chip
- **Initial state**
 - incremental: no components placed
 - complete-state: all components placed (e.g. randomly, manually)
- **Successor function (operators)**
 - incremental: place components, route wire
 - complete-state: move component, move wire
- **Goal test**
 - all components placed
 - components connected as specified
- **Path cost**
 - maybe complex
 - distance, capacity, number of connections per component

Real World Problems

Robot Navigation



- **States**
 - locations
 - position of actuators
- **Initial state**
 - start position (dependent on the task)
- **Successor function (operators)**
 - movement, actions of actuators
- **Goal test**
 - task-dependent
- **Path cost**
 - maybe very complex
 - distance, energy consumption

Real World Problems

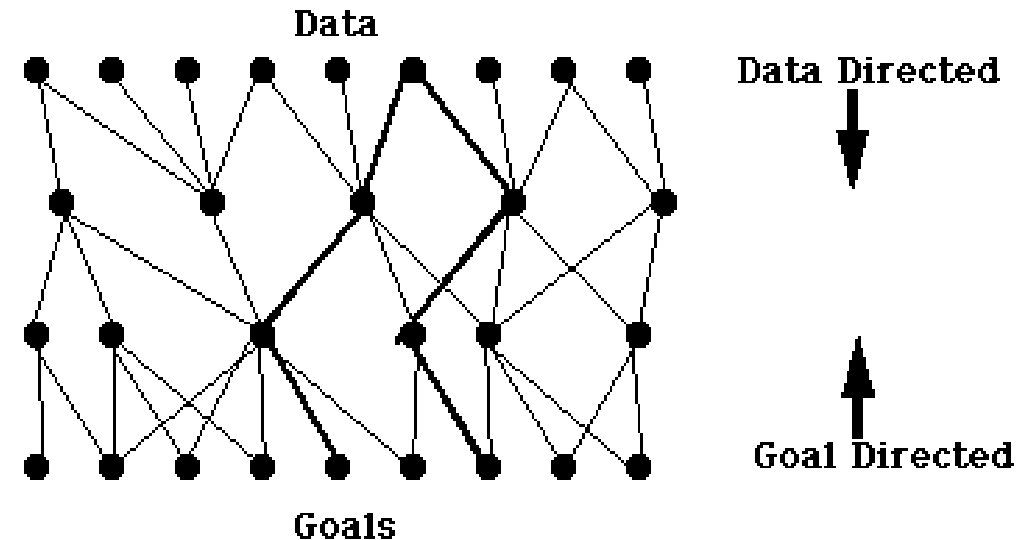
Automatic Assembly Sequencing



- **States**
 - location of components
- **Initial state**
 - no components assembled
- **Successor function (operators)**
 - place component
- **Goal test**
 - system fully assembled
- **Path cost**
 - number of moves

Search Strategies

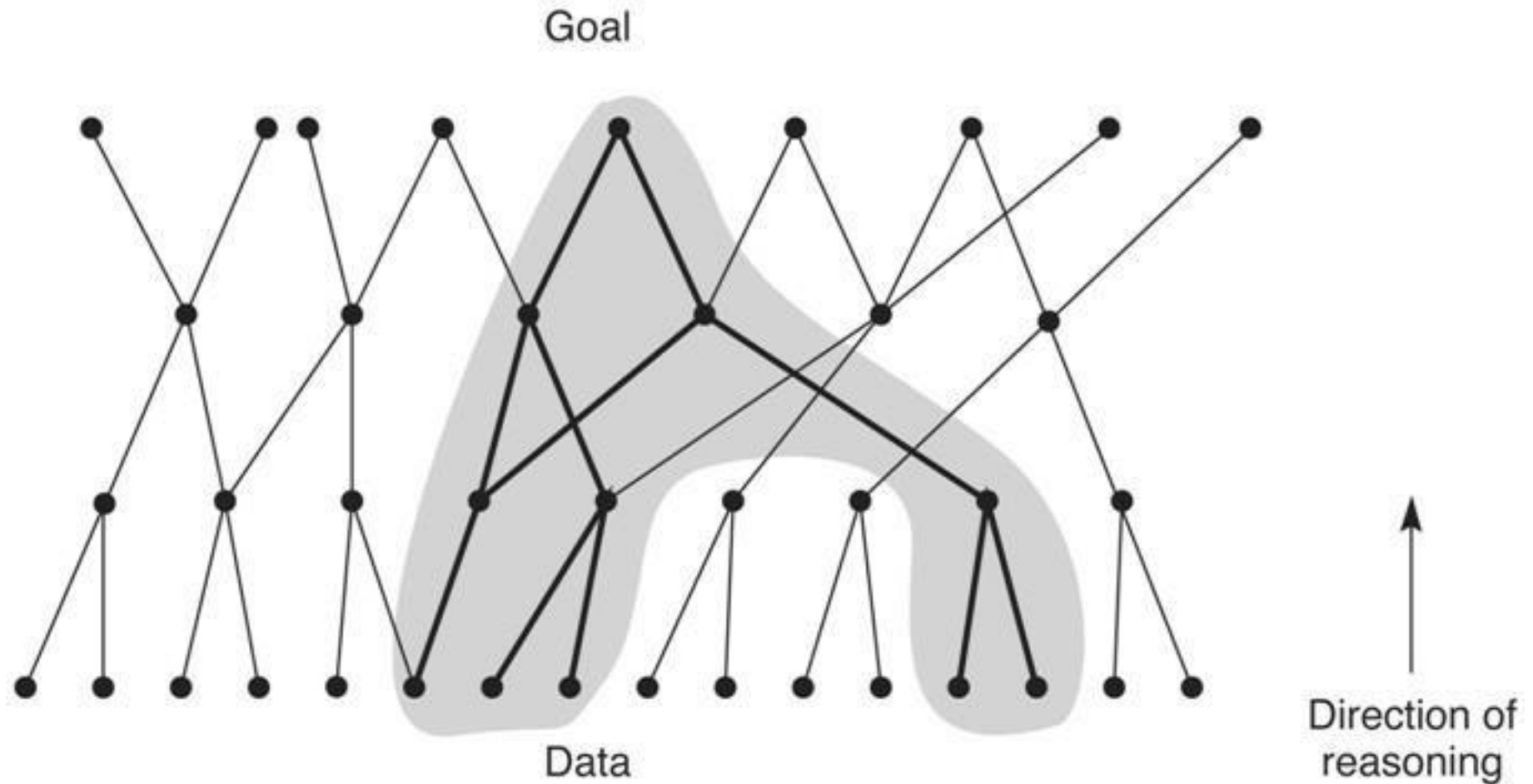
- The objective of search procedure is to discover a path through a problem spaces from an initial configuration to a goal state. There are two directions in which a search could proceed:
 - *Forward (from that initial states)*
 - *Backward (from the goal states)*



Forward Search

- Data-Directed / Data Driven Reasoning / Forward Chaining
- This search starts from available information and tries to draw conclusion regarding the situation or the goal attainment. This process continues until (hopefully) it generates a path that satisfies the goal condition.

Forward Search



Backward Search

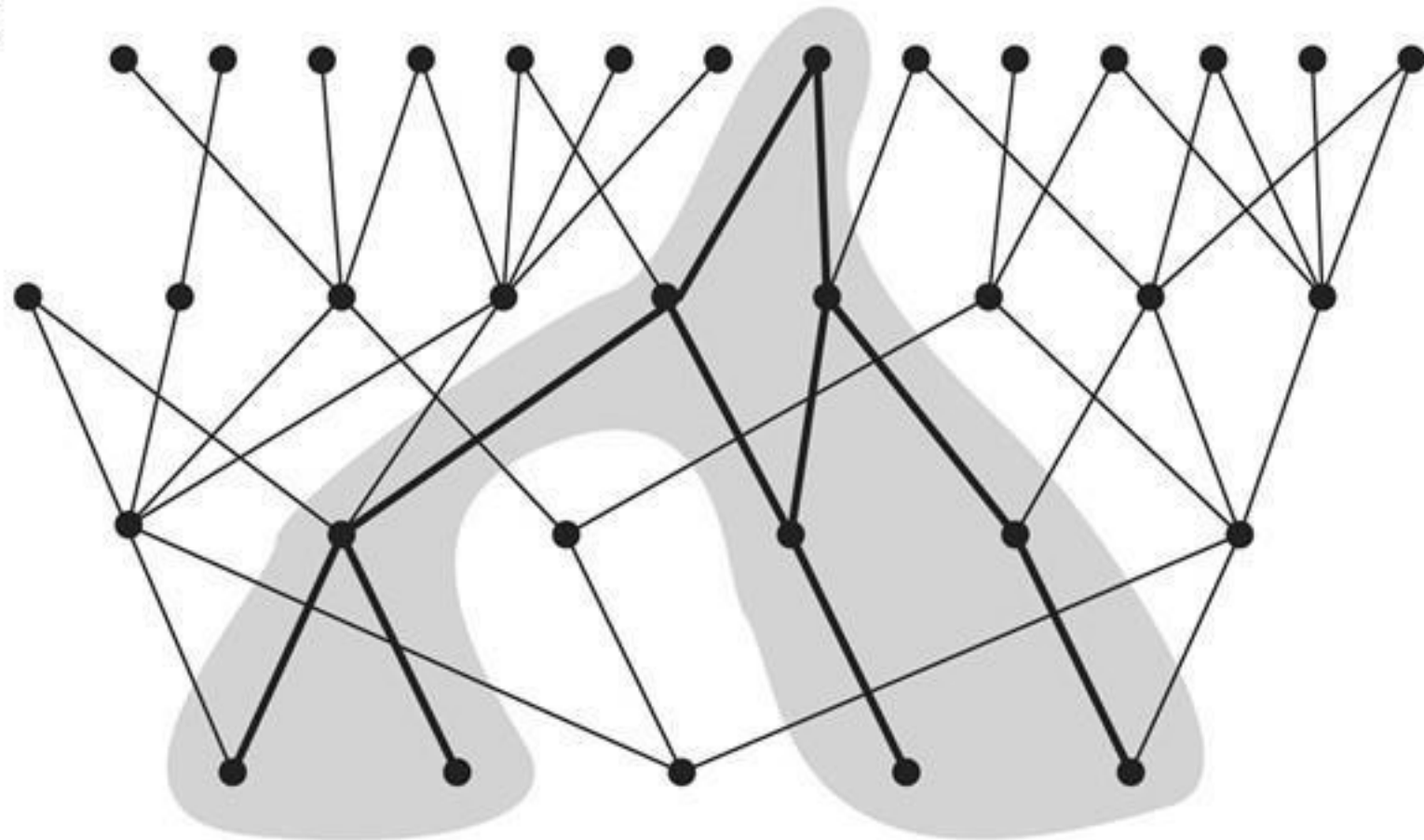
- Goal-Directed / Goal Driven Reasoning / Backward Chaining
- This search starts from expectations of what the goal is or what is to happen (hypothesis), then it seeks evidence that supports (or contradicts) those expectations (or hypothesis).
- The problem solver begins with the goal to be solved, then finds rules or moves that could be used to generate this goal and determine what conditions must be true to use them.
- These conditions become the new goals, sub goals, for the search. This process continues, working backward through successive sub goals, until (hopefully) a path is generated that leads back to the facts of the problem.

Backward Search

Direction of
reasoning



Goal



Data

Search Strategies

- Goal-driven search uses knowledge of the goal to guide the search. Use goal-driven search if;
 - *A goal or hypothesis is given in the problem or can easily be formulated.*
 - *There are a large number of rules that match the facts of the problem and thus produce an increasing number of conclusions or goals. (inefficient)*
 - *Problem data are not given but must be acquired by the problem solver. (impossible)*

Search Strategies

- Data-driven search uses knowledge and constraints found in the given data to search along lines known to be true. Use data-driven search if:
 - *All or most of the data are given in the initial problem statement.*
 - *There are a large number of potential goals, but there are only a few ways to use the facts and the given information of a particular problem.*
 - *It is difficult to form a goal or hypothesis.*

Evaluating Search Strategies

- **Completeness**

- *Guarantees finding a solution whenever one exists*

- **Time Complexity**

- *How long (worst or average case) does it take to find a solution?*

- **Space Complexity**

- *How much space is used by the algorithm?*

- **Optimality/Admissibility**

- *If a solution is found, is it guaranteed to be an optimal or highest quality solution among several different solutions? For example, is it the one with minimum cost?*

Questions

1. What is a problem-solving agent, and what are its essential components?
2. Discuss the role of problem-solving agents in artificial intelligence.
3. How do problem-solving agents represent problems internally?
4. Explain the importance of problem representation in the efficacy of problem-solving agents.
5. What is the significance of goal formulation in problem-solving agents?
6. How do problem-solving agents determine and prioritize goals?
7. Describe the role of search algorithms in problem-solving agents.
8. Compare and contrast different search algorithms commonly used by problem-solving agents.
9. What are the essential components of problem formulation in AI?
10. How does problem formulation differ from problem-solving?
11. Explain the difference between an undirected graph and a directed graph.
12. What is a weighted graph, and how does it differ from an unweighted graph?
13. How is an adjacency matrix used to represent a graph?
14. Define a path in the context of graph theory.
15. What is path cost, and how is it determined in a graph?
16. Explain the concept of the shortest path. How is it calculated in a graph?
17. What are the basic steps involved in solving a problem by searching?
18. Discuss the categories of problems in AI that can be solved using search techniques.
19. Provide examples of single-agent path-finding problems.
20. How do search strategies vary when solving single-agent path-finding problems?

Questions

21. What distinguishes two-player games from other types of problems in AI?
22. Explain how search algorithms can be applied to two-player games like chess or tic-tac-toe.
23. Define constraint satisfaction problems (CSPs) and provide examples.
24. How do search strategies differ when solving CSPs compared to path-finding problems?
25. Describe the concept of a problem space in AI.
26. What is the difference between a problem space and a problem instance?
27. How are states represented in the context of problem-solving agents?
28. Provide examples of state representations in common AI problems like tic-tac-toe or the 8-puzzle.
29. Discuss the challenges involved in solving real-world problems using AI techniques.
30. Provide examples of real-world problems that can be addressed through problem-solving agents.
31. Compare and contrast forward search and backward search strategies.
32. What factors should be considered when evaluating the effectiveness of search strategies in AI?
33. How can the efficiency and effectiveness of search strategies be evaluated in AI?
34. Discuss the importance of heuristic functions in evaluating search strategies.

Programming Tasks

1. Implement a class to represent an undirected graph using adjacency lists.
2. Implement a class to represent a directed graph using adjacency matrices.
3. Write a function to find all possible paths between two nodes in an undirected graph.
4. Implement Dijkstra's algorithm to find the shortest path between two nodes in a weighted graph.
5. Create a class representing a problem-solving agent capable of solving single-agent path-finding problems using various search algorithms.
6. Develop a problem-solving agent capable of playing tic-tac-toe against a human player or another agent.
7. Implement a backtracking algorithm to solve the N-Queens problem, ensuring no two queens attack each other.
8. Write a program to solve Sudoku puzzles using constraint satisfaction techniques.
9. Design a graphical user interface (GUI) for visualizing the search process of various algorithms on a graph.
10. Implement a web-based application where users can input a problem instance (e.g., graph, puzzle) and visualize the solution process step by step.